

DOCKER DEVOPS

HANDS-ON LAB

End-to-End Practical Guide for macOS and Windows

Application: Flask Web Service + Redis

Default host port in this revised guide: 8081

Lab goal

Build a Docker image, run and inspect containers, publish a port, understand the container writable layer, create persistent storage, understand registry operations, and deploy the two-service application using Docker Compose.

Important revision

All manual web commands now use host port 8081. Redis commands are kept on a single line to prevent shell line-break errors. Recovery steps are included for port conflicts, container-name conflicts, failed containers, and commands accidentally split across lines.

1. What You Will Build

You will use a small but realistic application: a Flask web service plus Redis. The Flask service exposes a home page, a health endpoint, and a Redis-backed page-view counter. Docker Compose runs both services together.

1.1 Final architecture

```
Browser
|
v
localhost:8081 -> Web container :5000
                  |
                  | Compose network
                  v
                Redis :6379
                  |
                  v
            redis-data volume
```

1.2 What each part demonstrates

Item	Why you are using it
Dockerfile	Build an application image from instructions.
Image	Reusable, layered artifact used to create containers.
Container	Running instance of an image.
Port publishing	Expose Flask container port 5000 through host port 8081.
docker exec / logs	Inspect and debug a running container.
Writable layer	Demonstrate that non-volume data is tied to the container.
Named volume	Keep Redis data outside the disposable container layer.
Registry	Tag and optionally push an image for distribution.
Compose	Define and run multiple services as one application stack.

Lab rule

Do not skip checkpoints. If a command fails, fix the problem before continuing. Never keep retrying docker run with the same container name without checking `docker ps -a` or removing the failed container.

2. Prerequisites and Installation

Use Docker Desktop with Linux containers. Docker Desktop includes Docker Engine, the Docker CLI, and Docker Compose, so you do not need a separate Compose installation.

2.1 macOS

- Use a currently supported macOS release and at least 4 GB RAM. Docker currently recommends Rosetta 2 on Apple-silicon Macs for some optional tooling, although it is not strictly required for normal Docker Desktop use.

2.1.1 Download Docker Desktop

Open the official Docker Desktop for Mac page and choose Apple silicon or Intel as appropriate.

2.1.2 Install

Open Docker.dmg and drag Docker to Applications.

2.1.3 Start Docker Desktop

Open Applications -> Docker and wait until Docker Desktop reports that it is running.

2.1.4 Verify the CLI

```
docker --version
docker compose version
docker info
```

2.1.5 Run the standard test container

```
docker run hello-world
```

Checkpoint PASS

The hello-world container should complete successfully. If it fails, use Section 16 before continuing.

2.2 Windows

For this lab, use Docker Desktop with Linux containers and the WSL 2 backend. Docker currently documents WSL 2 as the default/recommended route for most Windows users.

2.2.1 Check WSL

```
wsl --version
```

2.2.2 Install or update WSL if needed

Open PowerShell as Administrator only for this setup step.

```
wsl --install  
wsl --update
```

If Windows asks you to restart, restart and then run `wsl --version` again.

2.2.3 Install Docker Desktop

Download Docker Desktop for Windows and use the WSL 2 backend when offered.

2.2.4 Start Docker Desktop

Open Docker Desktop. In Settings, verify the WSL 2 based engine is enabled when the option is available. If you use a WSL distribution, enable it under Settings -> Resources -> WSL Integration.

2.2.5 Use Linux containers

This lab uses Linux images. If Docker Desktop is in Windows-container mode, switch to Linux containers from the Docker menu.

2.2.6 Verify the CLI

```
docker --version  
docker compose version  
docker info  
docker run hello-world
```

Checkpoint PASS

hello-world should complete successfully before starting the project. If not, use Section 16.

3. Get the Lab Files Ready

Use the supplied docker-devops-lab folder. You do not need Python, Flask, or Redis installed directly on your computer for this lab.

3.1 Folder structure

```
docker-devops-lab/  
├─ app/  
│   └─ app.py  
├─ .dockerignore  
├─ Dockerfile  
├─ compose.yaml  
├─ requirements.txt  
├─ README.md  
└─ START-HERE.txt
```

3.2 macOS setup

```
cd ~/Downloads/docker-devops-lab && pwd && ls
```

The exact folder may differ. The important point is that your terminal is inside docker-devops-lab before docker build or docker compose commands.

3.3 Windows PowerShell setup

```
cd "$HOME\Downloads\docker-devops-lab"  
Get-Location  
Get-ChildItem
```

3.4 Windows with WSL

You may use the same Docker commands from a WSL terminal with Docker Desktop integration enabled. For WSL-based development, keeping the project in the Linux filesystem can avoid cross-file system performance issues.

Windows filename warning

Make sure the file is exactly named Dockerfile, not Dockerfile.txt. Make sure compose.yaml is exactly compose.yaml. In File Explorer, turn on View -> Show -> File name extensions to verify names.

4. Practical 1 - Inspect the Project Before Building

The aim is to understand what Docker will build before asking Docker to build it.

4.1 Check the Dockerfile

```
cat Dockerfile
```

Windows PowerShell: use Get-Content Dockerfile

4.2 Understand the key instructions

- FROM python:3.12-slim - starts from a small Python runtime image.
- WORKDIR /app - sets the working directory inside the image.
- COPY requirements.txt . - copies dependency information into the image.
- RUN pip install --no-cache-dir -r requirements.txt - installs Flask and redis libraries.
- COPY app/ ./app/ - copies application source into the image.
- EXPOSE 5000 - documents the application port; it does not publish the port by itself.
- CMD ["python", "app/app.py"] - starts Flask when the container runs.

4.3 Check Compose

```
cat compose.yaml
```

Windows PowerShell: use Get-Content compose.yaml

5. Practical 2 - Build a Docker Image

5.1 Build the image

```
docker build -t docker-devops-lab:v1 .
```

The final dot is the build context: Docker uses the current project folder when resolving COPY instructions.

5.2 Confirm the image

```
docker images
```

5.3 Inspect image metadata

```
docker image inspect docker-devops-lab:v1
```

Checkpoint PASS

You should see docker-devops-lab with tag v1 in docker images.

6. Practical 3 - Run a Container and Publish a Port

Port standard for this guide

Use host port 8081 and container port 5000 throughout the lab. If 8081 is already occupied on a student machine, choose another free host port such as 8090 and change only the host-side port. Never change Flask port 5000 for this exercise.

6.1 Preflight: make sure host port 8081 is free

- macOS: run `lsof -nP -iTCP:8081 -sTCP:LISTEN`. No output normally means no visible macOS listener.
- Windows PowerShell: run `Get-NetTCPConnection -LocalPort 8081 -State Listen -ErrorAction SilentlyContinue`. No output normally means no visible Windows listener.

6.2 Run in foreground mode

```
docker run --name docker-lab-web-fg -p 8081:5000 docker-devops-lab:v1
```

6.3 Open the application

Open `http://localhost:8081` in your browser. Do not use `http://127.0.0.1:5000` or the container IP such as `172.17.0.2:5000`. Those are not the published host endpoint for this lab.

6.4 Stop the foreground container

Return to the terminal running the container and press `Ctrl+C`.

6.5 Confirm the container lifecycle

```
docker ps
docker ps -a
```

Checkpoint PASS

`docker ps` should not show `docker-lab-web-fg` as running. `docker ps -a` should show it with an Exited status.

7. Practical 4 - Detached Mode, Logs, and Lifecycle

7.1 Remove the previous stopped foreground container

```
docker rm docker-lab-web-fg
```

7.2 Run in detached mode

```
docker run -d --name docker-lab-web -p 8081:5000 docker-devops-lab:v1
```

7.3 Confirm it is running

```
docker ps
```

7.4 Open the application

Visit <http://localhost:8081>.

7.5 Read logs

```
docker logs docker-lab-web
```

7.6 Follow logs live

```
docker logs -f docker-lab-web
```

Press Ctrl+C to stop following the logs. This does not stop the container.

7.7 Stop and start the container

```
docker stop docker-lab-web
docker ps -a
docker start docker-lab-web
docker ps
```

7.8 Restart the container

```
docker restart docker-lab-web
```

Name-conflict recovery

If docker run says the container name is already in use, do not change ports first. Run `docker ps -a`, then `docker rm -f docker-lab-web` if you no longer need the existing lab container, and then rerun `docker run`.

8. Practical 5 - Inspect the Container Filesystem

8.1 Open a shell inside the running container

```
docker exec -it docker-lab-web sh
```

8.2 Inspect the application directory

```
pwd
ls
ls /app
cat /app/app/app.py | head -n 20
```

8.3 Exit the container shell

```
exit
```

The Docker command syntax is the same on macOS and Windows. Use Terminal, PowerShell, Windows Terminal, or a WSL terminal after Docker Desktop is running.

9. Practical 6 - Demonstrate the Disposable Container Layer

This experiment shows why important application data should not be stored only in a container writable layer.

9.1 Create a temporary file inside the container

```
docker exec docker-lab-web sh -c "echo temporary-data > /tmp/demo.txt"
```

9.2 Confirm the file exists

```
docker exec docker-lab-web sh -c "cat /tmp/demo.txt"
```

9.3 Remove the container

```
docker rm -f docker-lab-web
```

9.4 Create a new container from the same image

```
docker run -d --name docker-lab-web -p 8081:5000 docker-devops-lab:v1
```

9.5 Check whether the temporary file survived

```
docker exec docker-lab-web sh -c "cat /tmp/demo.txt"
```

Expected result

The final command should report that /tmp/demo.txt does not exist. That is the successful result: the new container was created from the same image, but the old container writable-layer data was not part of the image.

10. Practical 7 - Named Volume and Persistence

Now we store Redis data in a Docker-managed named volume so the data can survive container removal.

10.1 Stop and remove the current standalone web container

```
docker rm -f docker-lab-web
```

10.2 Create a named volume

```
docker volume create docker-lab-data
```

10.3 Check the volume

```
docker volume ls
```

10.4 Run Redis using the volume

```
docker run -d --name docker-lab-redis-volume -v docker-lab-data:/data redis:7-alpine redis-server --appendonly yes
```

10.5 Confirm Redis is ready

```
docker exec docker-lab-redis-volume redis-cli PING
```

Expected: PONG

10.6 Write a value into Redis

```
docker exec docker-lab-redis-volume redis-cli SET student_message "Docker volume works"
docker exec docker-lab-redis-volume redis-cli GET student_message
```

10.7 Remove the Redis container but keep the volume

```
docker rm -f docker-lab-redis-volume
```

10.8 Create another Redis container using the same volume

```
docker run -d --name docker-lab-redis-volume-2 -v docker-lab-data:/data redis:7-alpine redis-server --appendonly yes
```

10.9 Confirm Redis is ready again

```
docker exec docker-lab-redis-volume-2 redis-cli PING
```

10.10 Verify the data survived

```
docker exec docker-lab-redis-volume-2 redis-cli GET student_message
```

Expected result

The final command should return Docker volume works. If the command says the container is not running, run `docker ps -a` and `docker logs docker-lab-redis-volume-2`. If the name is already in use, remove the failed container with `docker rm -f docker-lab-redis-volume-2` and rerun Step 10.8 as one complete line.

11. Practical 8 - Image Tagging and Registry

A registry stores and distributes container images. Docker Hub is one example; cloud registries include Amazon ECR, Azure Container Registry, Google Artifact Registry, and GitHub Container Registry.

11.1 Tag the image for Docker Hub

```
docker tag docker-devops-lab:v1 YOUR_DOCKERHUB_USERNAME/docker-devops-lab:v1
```

11.2 List the tagged image

```
docker images
```

11.3 Optional: log in

```
docker login
```

Never paste credentials into the lab document, terminal screenshots, or classroom chat.

11.4 Optional: push

```
docker push YOUR_DOCKERHUB_USERNAME/docker-devops-lab:v1
```

11.5 Optional: pull on another machine

```
docker pull YOUR_DOCKERHUB_USERNAME/docker-devops-lab:v1
```

Checkpoint / classroom note

Tagging is required to understand the registry workflow. Push/pull can be demonstrated if Docker Hub access is available. Campus networks may block registry authentication.

12. Practical 9 - Deploy the Application with Docker Compose

Compose defines the web and Redis services together, places them on a private application network, and attaches a named volume to Redis.

12.1 Inspect compose.yaml

```
cat compose.yaml
```

Windows PowerShell: use Get-Content compose.yaml

12.2 Validate the Compose file

```
docker compose config
```

Checkpoint PASS

Docker should print the normalized Compose configuration. A YAML indentation error should be fixed before continuing.

12.3 Build and start the application

```
docker compose up -d --build
```

12.4 Check service status

```
docker compose ps
```

12.5 Open the web application

Visit <http://localhost:8081>.

12.6 Check the application health endpoint

```
curl http://localhost:8081/health
```

You can also open <http://localhost:8081/health> in a browser if curl is unavailable. Expected: a healthy response showing Redis connected.

12.7 Test the Redis-backed page view counter

Open <http://localhost:8081/counter> several times. The `page_views` value should increase because Redis stores the counter.

12.8 Inspect Compose logs

```
docker compose logs
docker compose logs web
docker compose logs redis
```

12.9 Follow web logs

```
docker compose logs -f web
```

Refresh the browser a few times, observe requests, then press Ctrl+C to stop following logs.

12.10 See running containers

```
docker ps
```

12.11 Stop and remove the Compose application

```
docker compose down
```

12.12 Start the stack again

```
docker compose up -d
```

12.13 Verify persistence

Open <http://localhost:8081/counter> again. The counter should continue from the stored Redis value because the redis-data named volume remains after docker compose down.

Checkpoint PASS

At this point you have built an image, published a port, run foreground and detached containers, inspected the filesystem, demonstrated disposable storage, used a named volume, understood registry tagging, and deployed a multi-container application with Compose.

13. What compose.yaml Means

Compose element	Meaning
services.web	The Flask application service.
build: .	Build the web image from the Dockerfile in the current directory.
ports: "8081:5000"	Maps host port 8081 to container port 5000.
environment	Tells Flask that Redis is reachable by service name redis.
depends_on: redis / service_healthy	Starts web after the Redis health check is healthy.
services.redis	Runs Redis from the official image.
healthcheck	Lets Compose know when Redis is ready to answer ping.
volumes / redis-data	Mounts named persistent storage into Redis /data.

Important technical point

Inside the Compose network, the web service should use redis as the Redis hostname, not localhost. localhost inside the web container refers to the web container itself.

14. Docker Swarm - Theory / Optional Demonstration

Swarm is included in the syllabus, but building a multi-node cluster is intentionally not required for this short practical. Understand the model: Compose defines a multi-container application; Swarm mode provides cluster management and service orchestration across Docker hosts.

```
Single Docker host
|
+-- Docker Compose

Multiple Docker hosts
|
+-- Swarm manager
|   +-- services / replicas
|
+-- Swarm workers
```

Important vocabulary: manager node, worker node, service, task, replica, desired state.

14.1 Optional single-node demonstration

14.1.1 Check current Swarm state

```
docker info --format "{{.Swarm.LocalNodeState}}"
```

14.1.2 Initialize Swarm if the state is inactive

```
docker swarm init
```

14.1.3 Create a service

```
docker service create --name swarm-web --publish 8085:80 nginx:alpine
```

14.1.4 Inspect the service

```
docker service ls
docker service ps swarm-web
```

14.1.5 Remove the service

```
docker service rm swarm-web
```

14.1.6 Leave single-node Swarm mode

```
docker swarm leave --force
```

Warning

Run the Swarm demonstration only if the instructor explicitly asks for it. It changes the Docker engine into Swarm mode and is not required for the core learning outcome.

15. Final Cleanup

15.1 Stop and remove the Compose application

```
docker compose down
```

15.2 List all containers

```
docker ps -a
```

15.3 Remove leftover lab containers if they still exist

```
docker rm -f docker-lab-redis-volume-2  
docker rm -f docker-lab-web
```

If Docker says a named container does not exist, that is fine. Continue to the next step.

15.4 List volumes

```
docker volume ls
```

15.5 Optional: remove lab volumes after you are completely finished

```
docker volume rm docker-lab-data  
docker volume rm redis-data
```

Do not remove volumes too early

The earlier persistence demonstrations depend on the named volumes. Only remove them after the practical and all verification steps are complete.

16. Troubleshooting - Common Student Errors

Symptom	What to do
docker: command not found / not recognized	Start Docker Desktop. On Windows, verify WSL 2 or the selected backend and then open a new terminal.
Cannot connect to the Docker daemon	Docker Desktop is not running or the engine is still starting. Open Docker Desktop and wait; rerun docker info.
docker compose: command not found	Use Docker Desktop and check docker compose version. Do not install an old standalone Compose tool for this lab.
WSL is not installed	Open Administrator PowerShell and run wsl --install, restart if asked, then verify with wsl --version.
Docker Desktop says Windows containers	Switch to Linux containers. The lab uses python:3.12-slim and redis:7-alpine.
address already in use / port is not available	The host port is occupied or unavailable. First inspect docker ps -a and the OS listener check. Use host 8081 by default; if unavailable choose 8090 and change only the host side of the mapping.
container name is already in use	The previous docker run created a container object even though startup later failed. Run docker ps -a, then docker rm -f <same-name>, and rerun the command.
docker exec says container is not running	Run docker ps -a and docker logs <container>. Start it with docker start <name> when appropriate, or remove/recreate the failed container.
web page opens at 127.0.0.1:5000 but not at localhost:8081	Use the published host port from -p HOST:CONTAINER. For this lab the browser endpoint is localhost:8081, while Flask listens on container port 5000.
Browser shows connection refused	Run docker ps, confirm the container is Up, confirm the host port mapping, and check docker logs. For this revised guide use http://localhost:8081.
Docker build fails on COPY	Make sure the terminal is inside docker-devops-lab and that Dockerfile, requirements.txt, and app/app.py exist with the exact names.
Dockerfile.txt on Windows	Turn on file-name extensions in File Explorer and rename Dockerfile.txt to Dockerfile.
YAML parse error in compose.yaml	YAML uses spaces, not tabs. Run docker compose config and compare the file with the supplied compose.yaml.
/health returns 503 or Redis unavailable	Run docker compose ps and docker compose logs redis / docker compose logs web. The revised Compose file waits for Redis health; ensure REDIS_HOST is redis, not localhost.
Redis container exits immediately	Check docker logs docker-lab-redis-volume-2. Make sure redis-server and --appendonly yes are separated by a space on the same command line.
zsh: command not found: server	The Redis command was accidentally split into two shell commands, e.g. redis- then Enter then server. Remove the failed container and rerun the complete command on one line.
Data disappeared after container removal	That is expected for data stored only in the container writable layer. Use a named volume for persistent data.
compose stack works but counter resets after full reset	docker compose down normally removes containers and networks but keeps named volumes. If you also removed redis-data, the previous counter is gone by design.
Permission or file access problems	Keep the project in a normal user-owned folder. For WSL, a project inside the Linux filesystem can avoid cross-filesystem quirks.

Golden recovery sequence

When stuck: docker ps -a -> docker logs <container> -> docker inspect <container> -> remove the failed container if needed -> rerun the command with a unique name and a known free host port. Do not change multiple variables at once.

17. Diagnostic Command Set

Use these commands before changing random settings. They show what Docker is currently doing.

```
docker info
docker version
docker ps
docker ps -a
docker images
docker volume ls
docker network ls
docker logs <container-name>
docker inspect <container-name>
docker compose ps
docker compose logs
```

17.1 Port checks

macOS:

```
lsof -nP -iTCP:8081 -sTCP:LISTEN
```

Windows PowerShell:

```
Get-NetTCPConnection -LocalPort 8081 -State Listen -ErrorAction SilentlyContinue
```

18. Student Check-off

- ☐ Docker Desktop installed and running.
- ☐ `docker --version` works.
- ☐ `docker compose version` works.
- ☐ `docker run hello-world` works.
- ☐ `docker-devops-lab:v1` image built successfully.
- ☐ Foreground container ran and was accessed through `http://localhost:8081`.
- ☐ Foreground vs detached mode demonstrated.
- ☐ `docker logs` and `docker exec` used.
- ☐ Temporary container filesystem behavior observed.
- ☐ Named volume created and persistence verified.
- ☐ Registry tag created; push/pull understood or demonstrated.
- ☐ `docker compose config` completed successfully.
- ☐ `docker compose up -d --build` completed successfully.
- ☐ `/health` returned a healthy response with Redis connected.
- ☐ `/counter` increased across requests.
- ☐ Compose logs inspected.
- ☐ `docker compose down` completed.

19. Official References

- [Install Docker Desktop on Mac](#)
- [Install Docker Desktop on Windows](#)
- [Docker Desktop WSL 2 backend](#)
- [Docker Compose installation overview](#)
- [Publishing and exposing ports](#)
- [Docker volumes](#)
- [Docker Swarm mode](#)